

Dynamically Refresh Spring Cloud Bus

Issue: when we update the config server then the client does not update we need to restart the client by ourselves.

- Request end point to see the config server from url

>> <http://localhost:8888/configdemo/dev>

- Configdemo: is the client server name
- Dev : is what we want to retrieve
- 8888: is the server port configuration server



The screenshot shows a web browser with the address bar set to `localhost:8888/configdemo/dev`. The page displays a JSON configuration for the 'dev' profile. The configuration includes a 'name' field set to 'configdemo', a 'profiles' array containing 'dev', and 'propertySources' which list two configuration files: 'classpath:/config/configdemo-dev.yml' and 'classpath:/config/configdemo.yml'. Each file entry includes a 'source' object with 'build.id', 'build.version', 'build.name', and 'build.type' fields.

```
{
  "name": "configdemo",
  "profiles": [
    "dev"
  ],
  "label": null,
  "version": null,
  "state": null,
  "propertySources": [
    {
      "name": "classpath:/config/configdemo-dev.yml",
      "source": {
        "build.id": 101,
        "build.version": "1.2.3",
        "build.name": "Native Dev-Build",
        "build.type": "Native Development Build"
      }
    },
    {
      "name": "classpath:/config/configdemo.yml",
      "source": {
        "build.id": 101,
        "build.version": "1.2.3",
        "build.name": "Native Default",
        "build.type": "Native Default Build"
      }
    }
  ]
}
```



The screenshot shows a web browser with the address bar set to `localhost:8888/configdemo/prod`. The page displays a JSON configuration for the 'prod' profile. The configuration includes a 'name' field set to 'configdemo', a 'profiles' array containing 'prod', and 'propertySources' which list two configuration files: 'classpath:/config/configdemo-prod.yml' and 'classpath:/config/configdemo.yml'. Each file entry includes a 'source' object with 'build.id', 'build.version', 'build.name', and 'build.type' fields.

```
{
  "name": "configdemo",
  "profiles": [
    "prod"
  ],
  "label": null,
  "version": null,
  "state": null,
  "propertySources": [
    {
      "name": "classpath:/config/configdemo-prod.yml",
      "source": {
        "build.id": 101,
        "build.version": "1.2.3",
        "build.name": "Native Production-Build",
        "build.type": "Native Production Build"
      }
    },
    {
      "name": "classpath:/config/configdemo.yml",
      "source": {
        "build.id": 101,
        "build.version": "1.2.3",
        "build.name": "Native Default",
        "build.type": "Native Default Build"
      }
    }
  ]
}
```

To resolve the issue, we can do it manually: with the actuator

1- Client

Add dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

URL to refresh: <http://localhost:8080/actuator/refresh>

The image shows a REST client interface for a POST request to `http://localhost:8080/actuator/refresh`. The request headers include `Content-Type: application/json`. Below the request, the response body is shown in JSON format:

```
[{"build.name": "Native Dev-Build12"}]
```

Below the REST client, a browser screenshot shows the response from `localhost:8080/build-info`:

Build ID: 101, Version: 1.2.3, Name: Native Dev-Build12

យើងធ្វើបានតែមួយទេ ចុះបើយើងមាន Microservice ច្រើនយើងហើយអាចជា Hard work ចឹង ហើយបានយើង ត្រូវប្រើប្រាស់ Spring Cloud Bus មកជួយវិញ ដោយធ្វើការ Brocase and subscriber.

Spring Cloud Bus

- Need message broker Rabbit MQ

RabbitMQ is a robust, open-source message broker that acts as a "post office" for your applications. It enables microservices and distributed systems to communicate asynchronously by accepting, storing, and forwarding binary blobs of data (messages).  RabbitMQ +1

- Run RabbitMQ with docker :

latest RabbitMQ 4.x

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:4-management
```

Step by step setup

1- Client

- Add dependencies: actuator, spring cloud bus

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- connect to rabbit mq -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

- Application.yml

```
spring:
  application:
    name: configdemo
  profiles:
    active: dev
  config:
    import: optional:configserver:http://localhost:8888

management:
  endpoints:
    web:
      exposure:
        include: refresh, health # Adds 'refresh' to the allowed endpoints

build:
  id: ${ID}
  version: ${VERSION}
  name: ${NAME}
```

+ ចំណាំ កន្លែង ភ្ជាប់ទៅ actuator

management:

endpoints:

web:

exposure:

include: refresh, health # Adds 'refresh' to the allowed endpoints

+ ចំណាំ កន្លែង ភ្ជាប់ទៅ

profiles:

active: dev

config:

import: optional:configserver:http://localhost:8888

```
1= spring:
2=   application:
3=     name: configdemo
4=   profiles:
5=     active: dev
6=   config:
7=     import: optional:configserver:http://localhost:8888
8
9= management:
10= endpoints:
11=   web:
12=     exposure:
13=       include: refresh, health # Adds 'refresh' to the allowed endpoints
14
15= build:
16=   id: ${ID}
17=   version: ${VERSION}
18=   name: ${NAME}
```

2- Config server: actuator, spring cloud bus

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

<!-- connect to rabbit mq -->

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

- configdemo-dev.yml

build:

id: 101
version: 1.2.3
name: "Native Dev-Vibol"
type: "Native Development Build"

spring:

rabbitmq:
 host: localhost
 port: 5672
 username: guest
 password: guest

management:

endpoints:
 web:
 exposure:
 include: "*"

3- Tesing

- url from the client : <http://localhost:8080/build-info>
- url for server end point : <http://localhost:8888/configdemo/dev>
- when we can from the server config-dev: after we can use the Post request to update the configuration from the server.

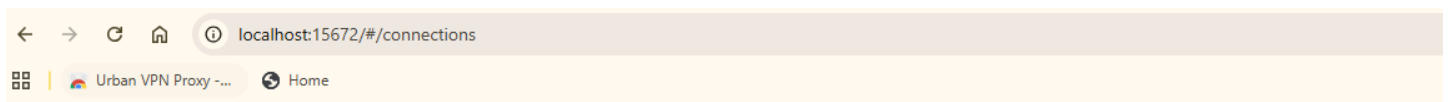
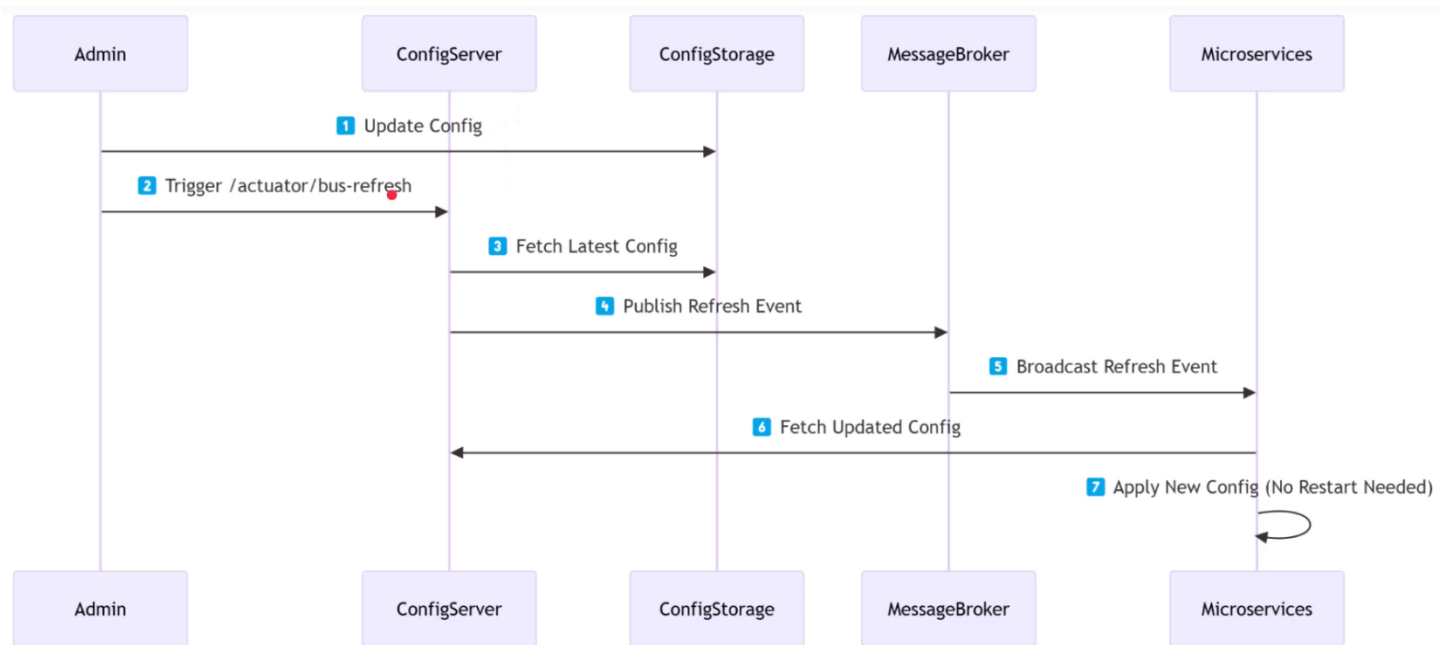
⇒ <http://localhost:8080/actuator/busrefresh>

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/actuator/busrefresh`. The request headers are visible, showing `Content-Type: application/json`. The response status is `204 No Content`, with a response time of 92 ms and a body size of 112 B. The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The Headers tab is active, showing 9 headers. The Body tab is also visible, showing the raw response.

Key	Value	Description	Bulk Edit	Presets	...
Content-Type	application/json				
Key	Value	Description			

Body: 204 No Content, 92 ms, 112 B. Save Response. Raw, Preview, Visualize.

4- Rabbit MQ is the message broker



RabbitMQ™ RabbitMQ 3.13.7 Erlang 26.2.5.16

[Overview](#) **[Connections](#)** [Channels](#) [Exchanges](#) [Queues and Streams](#) [Admin](#)

Connections

▼ All connections (2)

Pagination

Page **1** of 1 - Filter: ☐ Regex ?

Overview			Details			Network		+/-
Name	User name	State	SSL / TLS	Protocol	Channels	From client	To client	
172.20.0.1:42482 rabbitConnectionFactory#5acd7d1c:0	guest	running	○	AMQP 0-9-1	1	0 B/s	0 B/s	
172.20.0.1:47176 rabbitConnectionFactory#7ef60c02:0	guest	running	○	AMQP 0-9-1	1	0 B/s	2 B/s	

[HTTP API](#) [Documentation](#) [Tutorials](#) [New releases](#) [Commercial edition](#) [Commercial support](#) [Discussions](#) [Discord](#) [Plugins](#) [GitHub](#)